

Fundamentos de Aprendizaje Automático

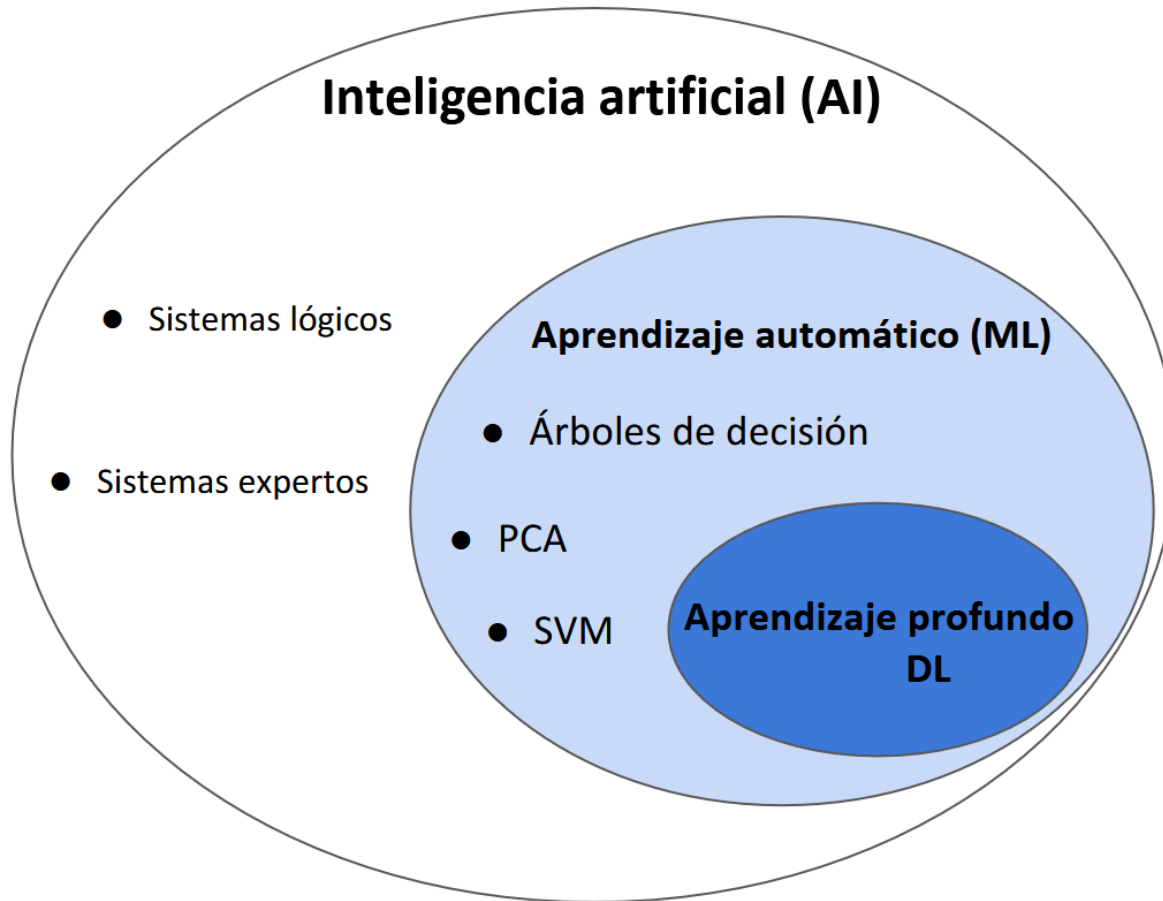
Lecture05 – Supervised Learning



Contenido

1. Supervised Learning
2. Fundamentos
3. Regresión lineal
4. Gradiente Descendente
5. Transformación de datos





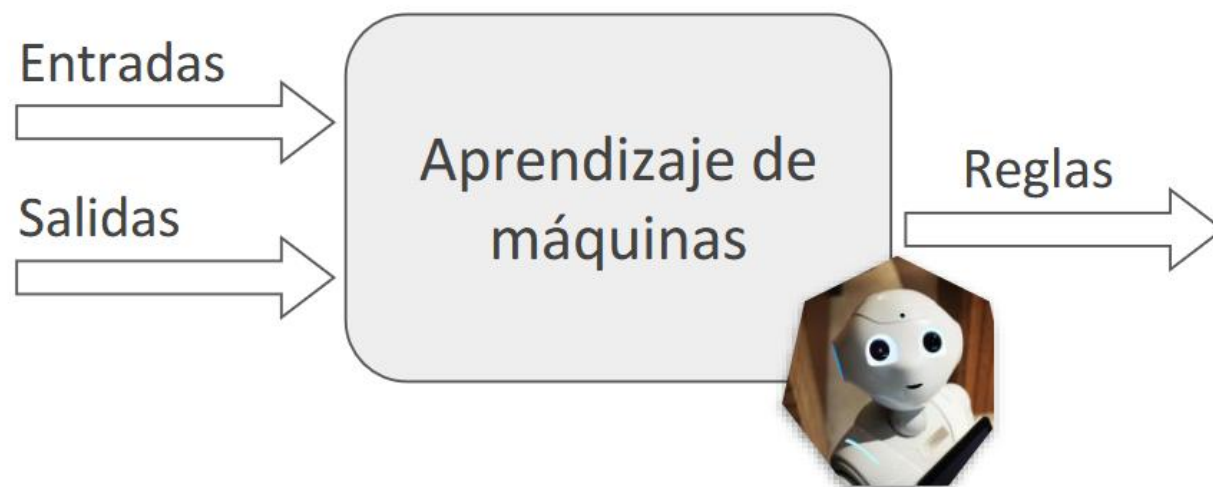
Inteligencia artificial: estudia los comportamientos inteligentes en dispositivos.

Machine learning: estudia los algoritmos que mejoran su rendimiento incorporando nuevos datos a un modelo existente.





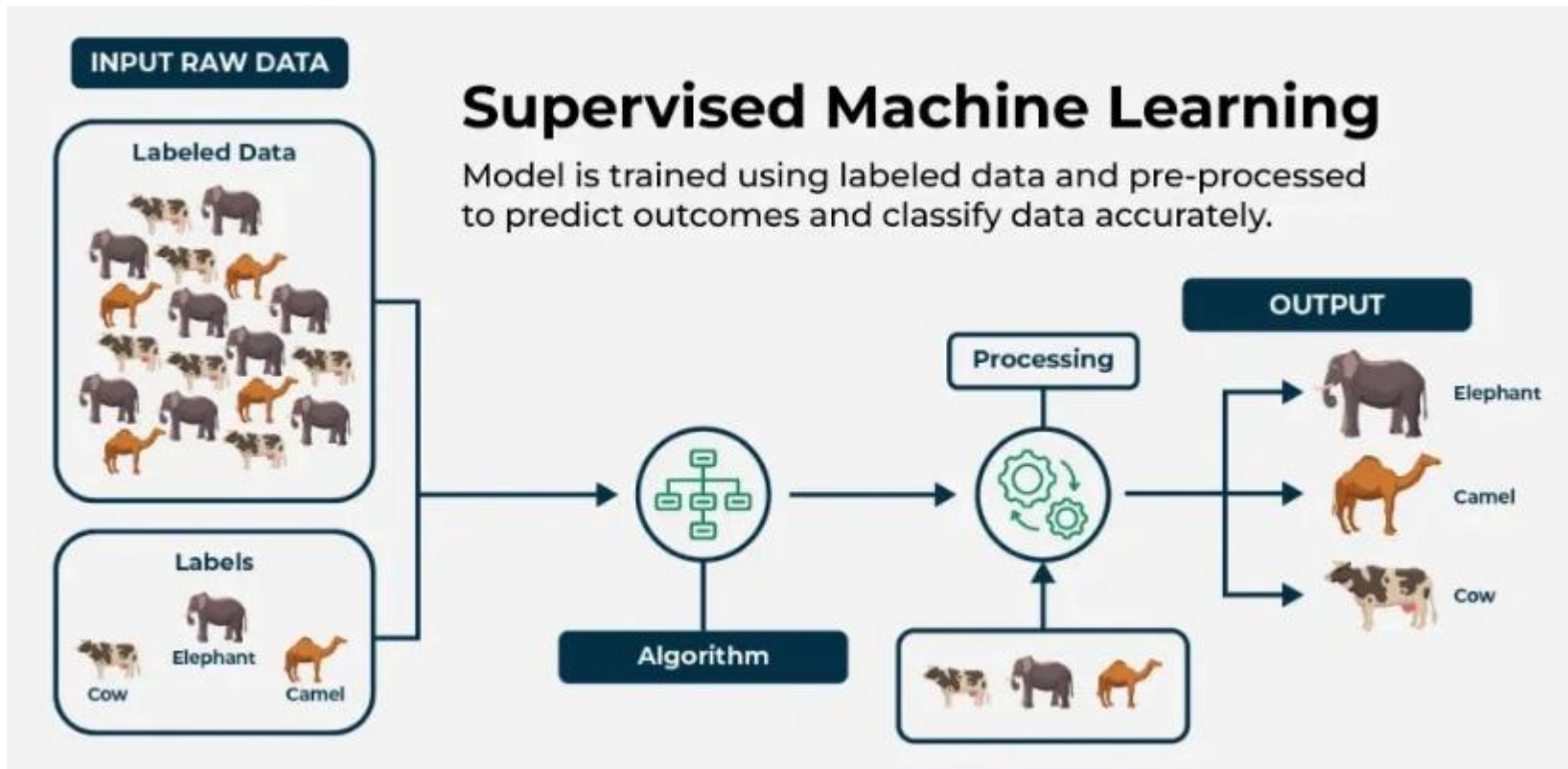
Se definen las reglas y el programador las implementa.



El algoritmo aprende las reglas en lugar de ser implementadas por un programador.



Supervised Learning



1. Comenzamos recolectando un conjunto de datos de muestras etiquetadas.
2. Entrenamos un modelo para que produzca predicciones precisas sobre este conjunto de datos.
3. Cuando el modelo ve datos nuevos y similares, también será preciso.

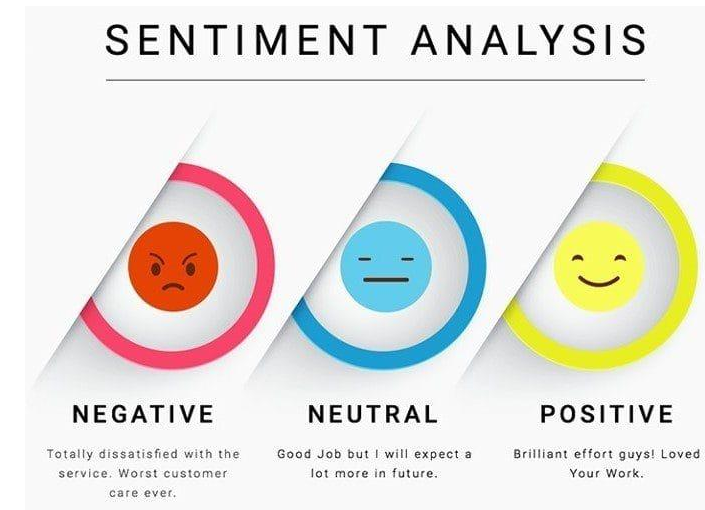
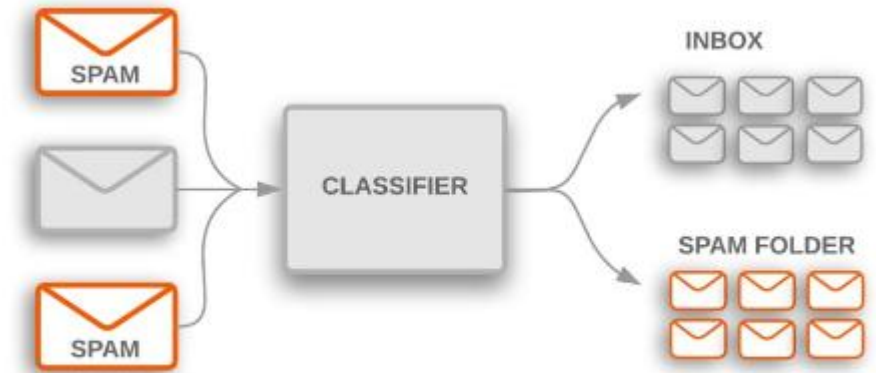
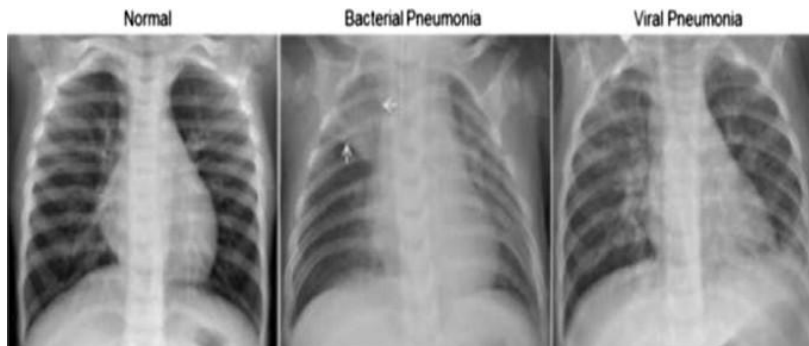


Supervised Learning

Aprendizaje Supervisado:

Aplicaciones:

- Clasificación de imágenes médicas.
- Detección de SPAM.
- Detección de objetos en la conducción autónoma.
- Reconocimiento de imágenes.
- Análisis de sentimiento.
- Predicción de tendencias numéricas



Notación matemática

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

Vector de características

$$\mathbf{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_d^{(i)} \end{bmatrix} \in \mathbb{R}^{d \times 1}$$

$$y^{(i)} \in \mathbb{R}$$

$$y^{(i)} \in [0, 1, \dots, C]$$

Matriz de características

$$\mathbf{X} = \begin{bmatrix} \dots & \mathbf{x}^{(1)\top} & \dots \\ \dots & \mathbf{x}^{(2)\top} & \dots \\ & \vdots & \\ \dots & \mathbf{x}^{(n)\top} & \dots \end{bmatrix} \in \mathbb{R}^{n \times d}$$

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

Aproximación de la relación

$$f(\mathbf{x}^{(i)}) = y^{(i)}$$

$$\mathcal{L}(f(\mathbf{x}^{(i)}), y^{(i)})$$



Regresión Lineal

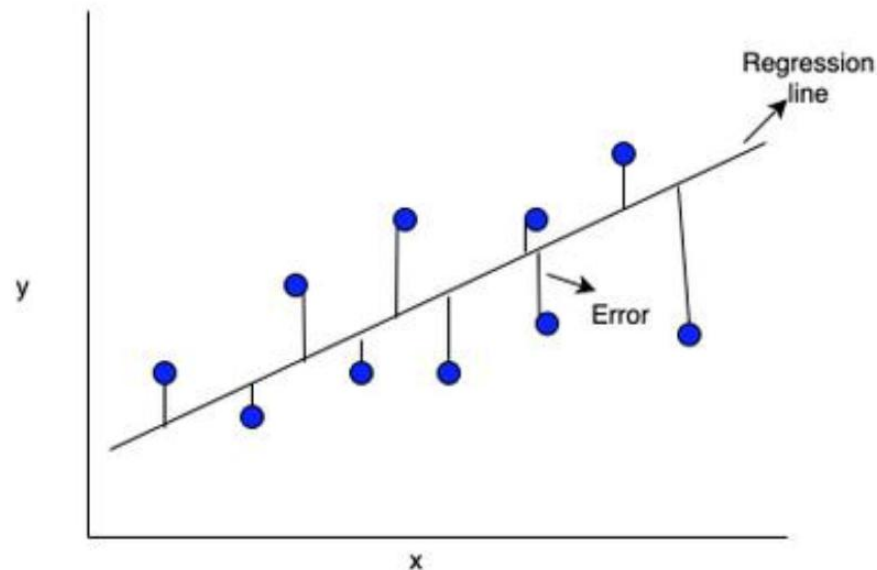
Definición:

Es un algoritmo de aprendizaje supervisado que busca encontrar la relación matemática entre una variable dependiente (y) y una o más variables independientes (x). Se usa para predecir valores numéricos, no categóricos.

Tipos:

Regresión Lineal Simple: Usa una sola variable x para predecir y (Ej: Años de experiencia $\rightarrow$ Salario).

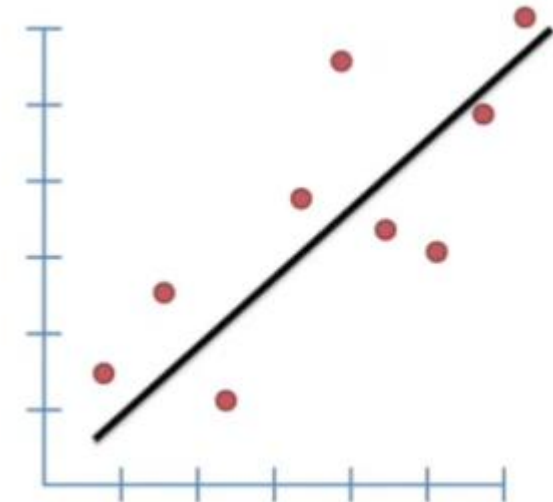
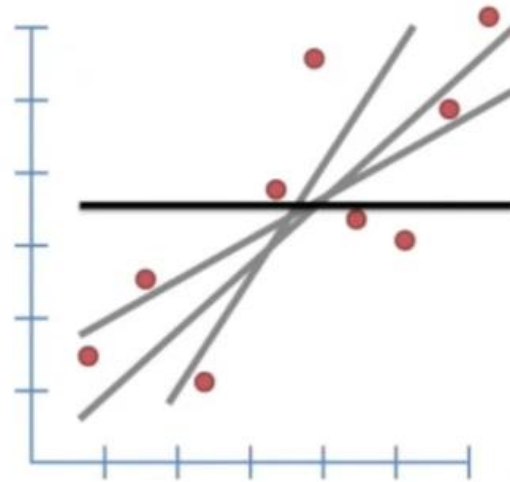
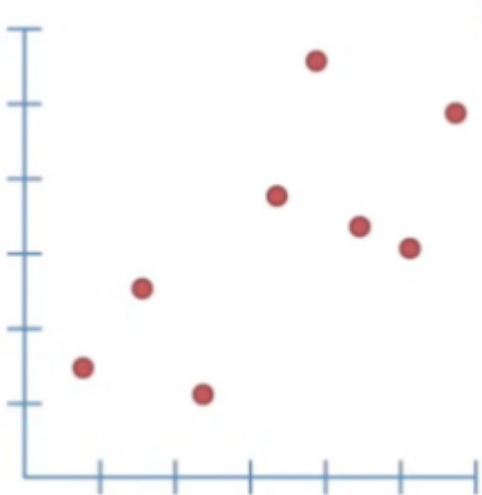
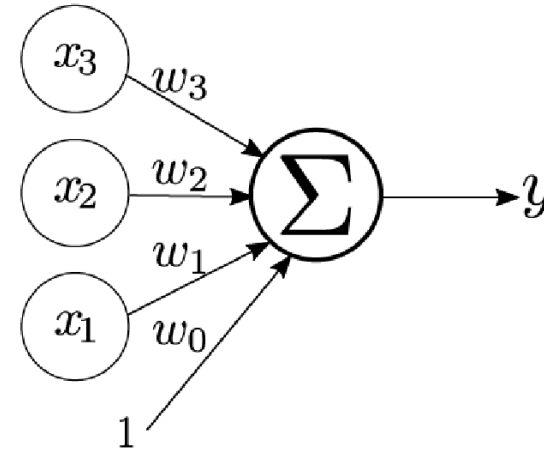
Regresión Lineal Múltiple: Usas varias variables $x_1, x_2, x_3, \dots$ Para predecir y . (Ej: Metros cuadrados + Ubicación + Antigüedad $\rightarrow$ Precio de casa).



Regresión Lineal

$$\hat{y} = \hat{f}(\mathbf{x}) = w_0 + \sum_{j=1}^d w_j \times x_j$$

$$\hat{y} = \mathbf{w}^T \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$$

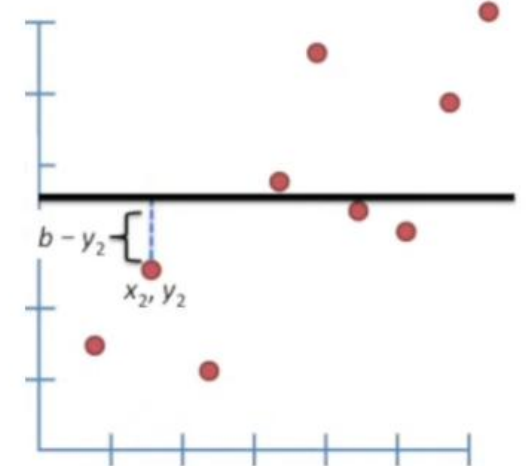
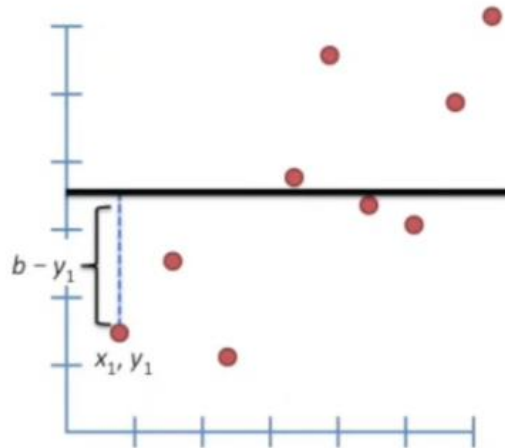
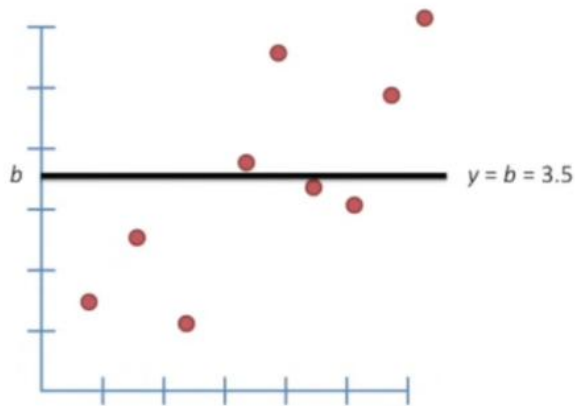


Regresión Lineal

Función de costo:

Una forma de medir que tan bien ajustado esta el modelo a los datos

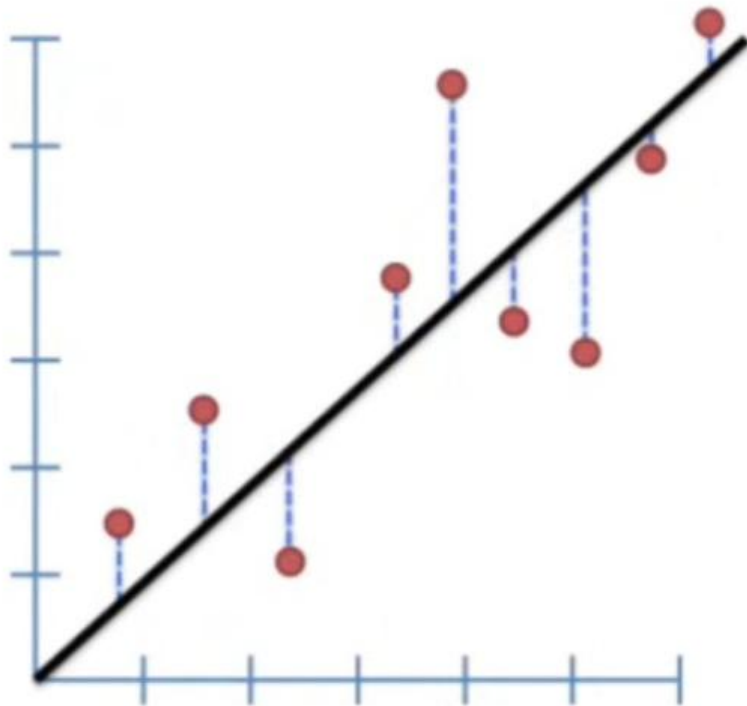
$$Cost_{\mathbf{w}} = \frac{1}{m} \sum_{i=1}^n \left(y_i - \mathbf{w}^T \begin{bmatrix} \mathbf{x}_i \\ 1 \end{bmatrix} \right)^2$$



Regresión Lineal

Función de costo:

Una forma de medir que tan bien ajustado esta el modelo a los datos



Suma de Cuadrados Residuales (SSR)

Es la suma total de los errores al cuadrado entre los valores predichos por la línea y los valores reales observados.

$$SSR = \sum_{i=1}^n ((wx_i + b) - y_i)^2$$

Error Cuadrático Medio (MSE)

Es el promedio de los errores al cuadrado; es el SSR dividido entre el número de datos n.

$$MSE = \frac{1}{n} \sum_{i=1}^n ((wx_i + b) - y_i)^2$$



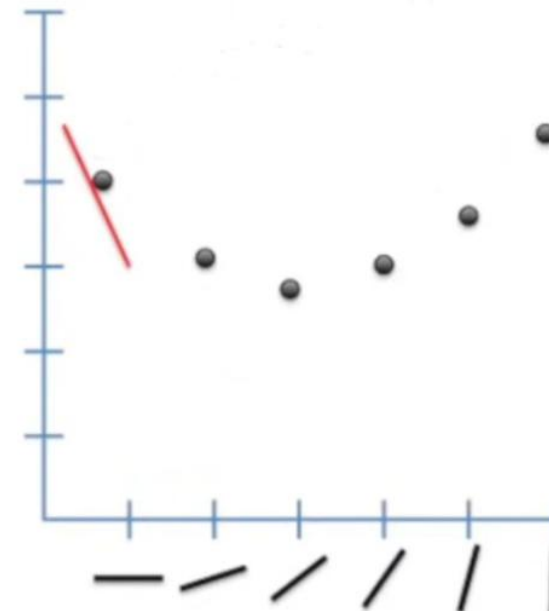
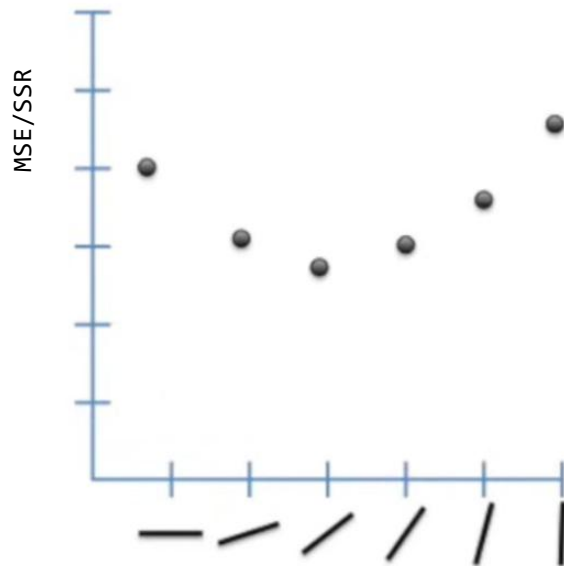
Regresión Lineal

Función de costo.

Como se definen entonces w y b ?

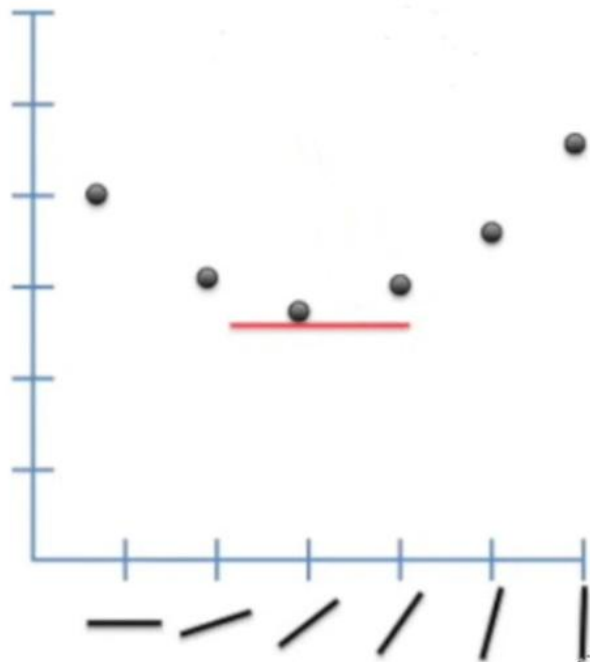
Dado que queremos encontrar los valores de a y b que den la menor suma posible, entonces este método se denomina **Mínimos Cuadrados**.

La respuesta es:
Derivamos la función.



Regresión Lineal

Función de costo:



$$\nabla f(\mathbf{r}) = \left(\frac{\partial f(\mathbf{r})}{\partial x_1}, \dots, \frac{\partial f(\mathbf{r})}{\partial x_n} \right)$$

En este tercer punto la derivada sería casi cero. Este proceso de encontrar la derivada lo realiza un algoritmo llamado **Gradiente Descendente**.

Descenso por el gradiente:

$$\mathbf{w} := \mathbf{w} - \alpha \times \nabla Cost_{\mathbf{w}}$$

$$\nabla Cost_{\mathbf{w}} = \frac{1}{m} X^T (\hat{y} - y)$$

Ecuación normal:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} (\mathbf{X}^T \mathbf{Y})$$

$$X = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \dots \\ 1 & x_{2,1} & x_{2,2} & \dots \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_{n,1} & x_{n,2} & \dots \end{bmatrix} \quad w = \begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \end{bmatrix}$$



Regresión Lineal

Función de costo.

Gradiente descendente

Algorithm Pseudocódigo del algoritmo del GD Modificado

```
1: Inicializar  $x_0$ 
2: Inicializar  $\alpha$ 
3: Inicializar convergencia
4: while  $\|\nabla f(x_0)\| > \text{convergencia}$  do
5:    $x_0 = x_0 - \alpha * \nabla f(x_0)$ 
6: end while
7:  $x^* \leftarrow x_0$ 
```

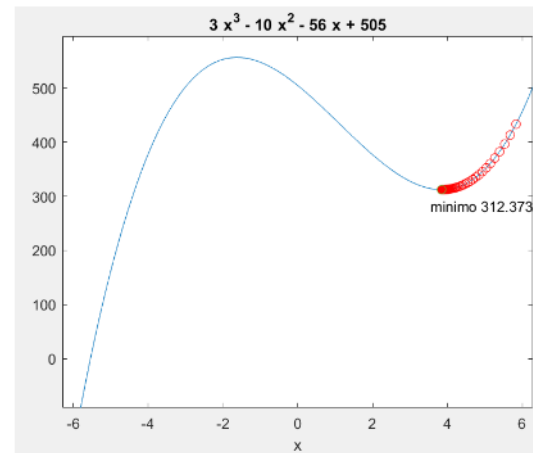
Características del algoritmo:

- X_0 : valor de arranque.
- Alfa: define el tamaño del paso (Learning Rate).
- Convergencia: número por debajo del cual consideramos que la derivada es cero.

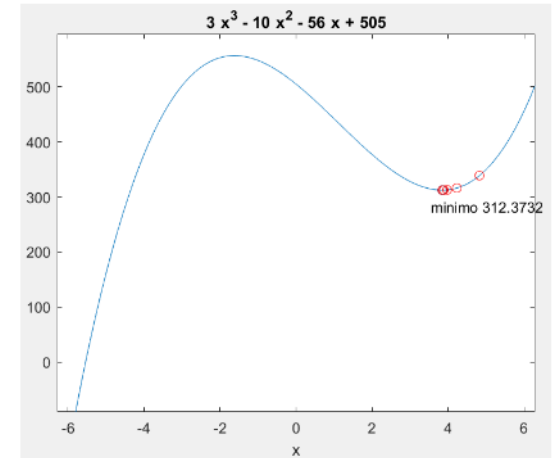
Norma de un vector:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$$

Alfa < 0.1



Alfa > 0.1



Regresión Lineal

Función de costo.

Como sabemos finalmente que tan bueno es el modelo?

Metricas de la regresión lineal:

Error Medio Absoluto (MAE): es el promedio de las distancias entre las predicciones y los valores reales.

Error = Real - Predicción

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$



Regresión Lineal

Función de costo.

Como sabemos finalmente que tan bueno es el modelo?

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Metricas de la regresión lineal:

R^2 = (Coeficiente de Determinación): calcula qué tan "bien" se ajusta la predicción con los datos reales.

"¿Qué tan mejor es mi modelo comparado con simplemente predecir el promedio?"

$R^2 = 1$ El modelo explica el 100% de la variabilidad de Y; todas las predicciones caen exactamente sobre los valores reales.

$R^2 = 0$ significa que el modelo predice igual de mal (o igual de bien) que simplemente usar el promedio de Y para todas las predicciones. No es que "no explique nada" en un sentido absoluto , es que no aporta nada *sobre lo que ya te daría la media.*

$R^2 = 0.85$ significa que hay una fuerte asociación estadística que tu modelo lineal logra capturar.

$R^2 < 0$ Que tu modelo es peor que simplemente predecir el promedio de Y para todos los casos. Es decir, tu línea de regresión ajusta tan mal que hasta un modelo "tonto" (la media) le gana.



Regresión Lineal

EJEMPLO PRACTICO Y EJERCICIOS



Transformación de datos

La mayoría de los algoritmos de Machine Learning regresión lineal, redes neuronales, SVM, KNN, entre otros– están contruidos sobre operaciones matemáticas: sumas, multiplicaciones, distancias, gradientes. Estos algoritmos simplemente no saben qué hacer con la palabra "rojo" o "Bogotá".

- Es un requisito técnico, no opcional.
- La forma en que transformas importa tanto como el hecho de transformar.
- Existen distintas técnicas para distintos casos.
- Afecta directamente el desempeño del modelo



Transformación de datos

Las preguntas clave que debes hacerte siempre es:

¿la variable tiene un orden natural? y ¿cuántas categorías tiene?

One-Hot Encoding

Cuando la variable no tiene orden (nominal) y vas a usar un modelo que sí interpreta magnitudes: regresión lineal/logística, SVM, KNN, redes neuronales

Es la opción "segura por defecto" para variables nominales con pocas categorías (regla general: menos de ~10-15 categorías).

Ciudad		Ciudad_Madrid	Ciudad_Paris	Ciudad_Londres
Madrid		1	0	0
Paris		0	1	0
Londres		0	0	1
Madrid		1	0	0



Transformación de datos

One-Hot Encoding

```
# Selecciona la columna a codificar
columna_a_codificar = df['Categoria']

# Aplica pd.get_dummies()
df_dummies_categoria = pd.get_dummies(columna_a_codificar, prefix='Categoria')

print("\nDataFrame de 'dummies' para la columna 'Categoria':")
print(df_dummies_categoria)

# Concatenar el DataFrame original con las nuevas columnas "dummy"
# Es buena práctica eliminar la columna original de texto después de la codificación
df_encoded_simple = pd.concat([df.drop('Categoria', axis=1), df_dummies_categoria], axis=1)
```



¿la variable tiene un orden natural? y ¿cuántas categorías tiene?

Label encoding

Cuando la variable no tiene orden, y vas a usar un modelo que no le importa el orden típicamente modelos de árboles (Decision Trees, Random Forest, XGBoost, LightGBM)

⚠ Es la técnica más mal utilizada porque parece la más simple, pero solo es segura con modelos basados en árboles.

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# 1. Datos de ejemplo
df = pd.DataFrame({'Nivel': ['Bajo', 'Medio', 'Alto', 'Bajo', 'Alto']})

# 2. Inicializar y aplicar el encoder
le = LabelEncoder()
df['Nivel_Encoded'] = le.fit_transform(df['Nivel'])

print(df)
```

	Nivel	Nivel_Encoded
0	Bajo	1
1	Medio	2
2	Alto	0
3	Bajo	1
4	Alto	0



¿la variable tiene un orden natural? y ¿cuántas categorías tiene?

Ordinal Encoding

Cuando la variable sí tiene un orden natural/jerárquico: nivel educativo, talla de ropa (S, M, L, XL), nivel de satisfacción (bajo, medio, alto), rangos de edad. Funciona bien con cualquier tipo de modelo, incluidos lineales, porque el orden numérico refleja un orden real en los datos.

Si los datos NO tienen orden (como "Rojo", "Verde", "Azul"), el Label Encoding puede confundir a modelos como la Regresión Lineal o Redes Neuronales, haciéndoles creer que el "Azul" (2) vale el doble que el "Verde" (1).



¿la variable tiene un orden natural? y ¿cuántas categorías tiene?

Ordinal Encoding

Cuando la variable sí tiene un orden natural/jerárquico: nivel educativo, talla de ropa (S, M, L, XL), nivel de satisfacción (bajo, medio, alto), rangos de edad. Funciona bien con cualquier tipo de modelo, incluidos lineales, porque el orden numérico refleja un orden real en los datos.

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# Datos de ejemplo
df = pd.DataFrame({
    'nivel_educativo': ['secundaria', 'universidad', 'primaria', 'maestria', 'secundaria']
})

# Definimos el orden jerárquico manualmente (de menor a mayor)
orden_jerarquico = ['primaria', 'secundaria', 'universidad', 'maestria']

encoder = OrdinalEncoder(categories=[orden_jerarquico])

df['nivel_educativo_encoded'] = encoder.fit_transform(df[['nivel_educativo']])

print(df)
```



¿la variable tiene un orden natural? y ¿cuántas categorías tiene?

Binary encoding

Cuando tienes una variable nominal (sin orden) con muchas categorías (alta cardinalidad) y quieres evitar la explosión de columnas de One-Hot, pero tampoco quieres el sesgo de orden falso de Label Encoding.

Es un punto intermedio: reduce dimensionalidad comparado con One-Hot, manteniendo cierta independencia entre categorías.

```
pip install category_encoders
```

```
import pandas as pd
import category_encoders as ce
```

```
# 1. Creamos un dataset con muchas categorías
```

```
data = {'Producto': ['Laptop', 'Mouse', 'Monitor', 'Teclado', 'Cable', 'Cámara', 'Micro', 'Silla']}
df = pd.DataFrame(data)
```

```
# 2. Inicializamos el codificador binario
```

```
encoder = ce.BinaryEncoder(cols=['Producto'])
```

```
# 3. Transformamos los datos
```

```
df_binary = encoder.fit_transform(df)
```

```
print("Dataset Original:")
```

```
print(df)
```

```
print("\nDataset con Binary Encoding:")
```

```
print(df_binary)
```

Dataset Original:

	Producto
0	Laptop
1	Mouse
2	Monitor
3	Teclado
4	Cable
5	Cámara
6	Micro
7	Silla

Dataset con Binary Encoding:

	Producto_0	Producto_1	Producto_2	Producto_3
0	0	0	0	1
1	0	0	1	0
2	0	0	1	1
3	0	1	0	0
4	0	1	0	1
5	0	1	1	0
6	0	1	1	1
7	1	0	0	0

Transformación de datos

¿la variable tiene un orden natural? y ¿cuántas categorías tiene?

Agrupación

Si tenemos una columna de "Ciudades" con 500 opciones, pero 450 de ellas solo aparecen una vez, agrúpalas todas en una nueva categoría llamada "Otros".

```
import pandas as pd

# 1. Creamos un dataset de ejemplo
data = {'Ciudad': ['Madrid', 'Madrid', 'Madrid', 'Bogotá', 'Bogotá', 'CDMX', 'Tokio', 'Oslo', 'París', 'Seúl']}
df = pd.DataFrame(data)

# 2. Definimos un umbral (threshold)
# En este caso, si la ciudad aparece menos de 2 veces, será "Otros"
umbral = 2
frecuencias = df['Ciudad'].value_counts()

# 3. Identificamos las categorías que no cumplen el umbral
categorias_infrecuentes = frecuencias[frecuencias < umbral].index

# 4. Reemplazamos esas categorías
df['Ciudad_Agrupada'] = df['Ciudad'].replace(categorias_infrecuentes, 'Otros')

print("Frecuencias originales:")
print(frecuencias)
print("\nDataset resultante:")
print(df)
```

Frecuencias originales:

Ciudad	
Madrid	3
Bogotá	2
CDMX	1
Tokio	1
Oslo	1
París	1
Seúl	1

Name: count, dtype: int64

Dataset resultante:

	Ciudad	Ciudad_Agrupada
0	Madrid	Madrid
1	Madrid	Madrid
2	Madrid	Madrid
3	Bogotá	Bogotá
4	Bogotá	Bogotá
5	CDMX	Otros
6	Tokio	Otros
7	Oslo	Otros
8	París	Otros
9	Seúl	Otros



Transformación de datos

Técnica	¿Requiere orden?	Mejor para	Cuidado con
Label Encoding	No (pero lo impone)	Modelos de árboles, variables nominales	Modelos lineales/distancia — introduce orden falso
Ordinal Encoding	Sí, orden real	Variables con jerarquía natural (tallas, niveles)	Definir mal el orden = error grave
One-Hot Encoding	No	Variables nominales, pocas categorías	Alta cardinalidad → explosión de columnas
Binary Encoding	No	Variables nominales, muchas categorías	Menos interpretable que One-Hot



Transformación de datos

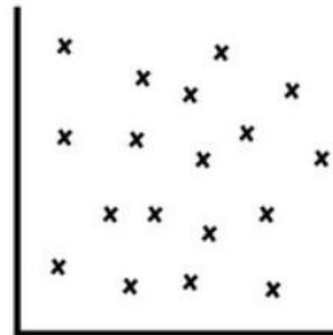
Correlación



Positive
Correlation



Negative
Correlation



No
Correlation

Valor de r	Interpretación
$r = 1$	Correlación positiva perfecta
$0,7 \leq r < 1$	Correlación positiva fuerte
$0,4 \leq r < 0,7$	Correlación positiva moderada
$0,1 \leq r < 0,4$	Correlación positiva débil
$r = 0$	Sin correlación lineal (Nula)
$-0,4 < r \leq -0,1$	Correlación negativa débil
$-0,7 < r \leq -0,4$	Correlación negativa moderada
$-1 < r \leq -0,7$	Correlación negativa fuerte
$r = -1$	Correlación negativa perfecta

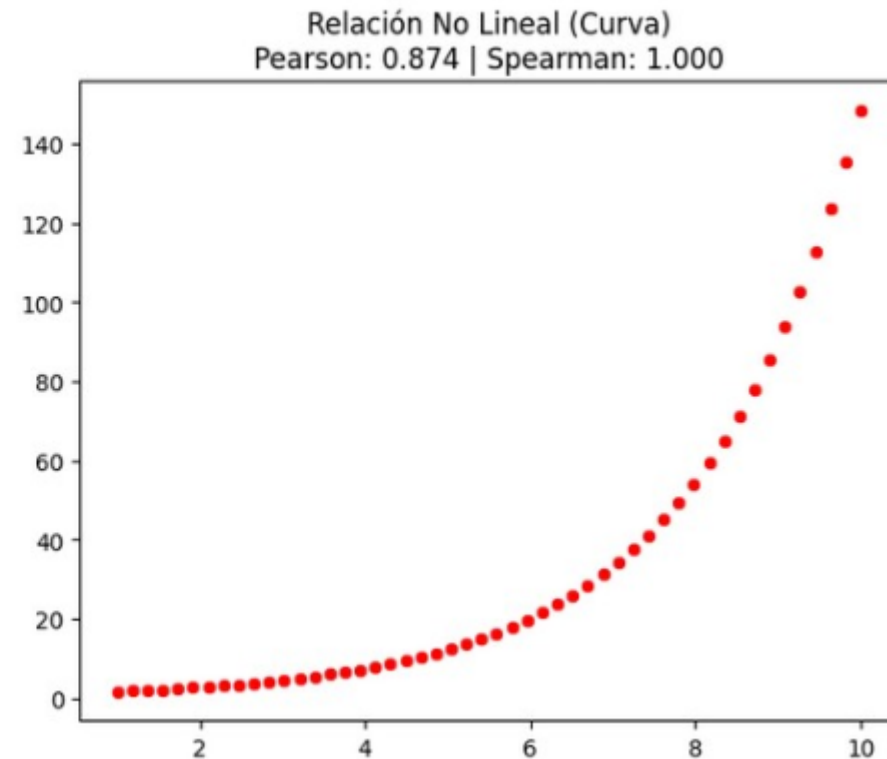
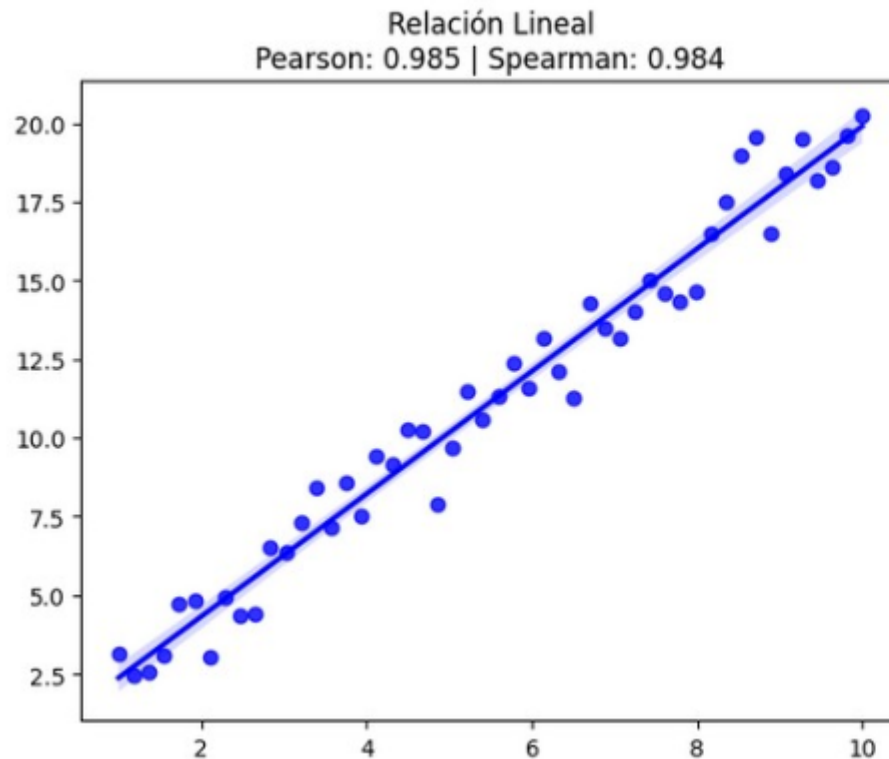


Transformación de datos

Correlación

Si hay mucha relación entre dos columnas y es lineal, por defecto se usa la correlación “Pearson”.

Si hay mucha relación pero no es lineal sino curva, se usa la correlación “Spearman”.



Correlación

¿Cuál de las dos columnas eliminar?

En principio, eliminar cualquiera de las dos. Sin embargo, se recomiendan los siguientes criterios:

- Eliminar la que sea más difícil de obtener: Si una columna viene de un sensor caro y la otra de un cálculo simple, quedarse con la simple.
- Eliminar la que tenga más valores nulos: Si la Columna A tiene un 5% de datos faltantes y la B está completa, borra la A.
- Eliminar la que sea menos "interpretable" (la mas confusa): Si una es "Edad" y la otra es "Días desde el nacimiento", quédate con "Edad" porque es más fácil de entender para un humano
- Si la correlación entre una columna y la variable objetivo (columna etiquetada) es muy cercana a 0, considerela muy seriamente a eliminar dicha columna.



Escalamiento de datos

Los modelos que se basan en distancias o gradientes son sensibles a la escala (KNN, K-Means, SVM, Redes Neuronales, regresión.); los modelos de árboles, no.

PORQUE ES IMPORTANTE ESCALAR CARACTERISTICAS?

Si una variable como Edad va de 0 a 100 y otra como Salario Anual va de 0 a 1,000,000, el algoritmo de ML pensará que el salario es 10,000 veces más importante solo porque sus números son más grandes.

El escalado pone a ambas variables en igualdad de condiciones.



Escalamiento de datos

¿Qué pasa si NO se escala?

Algoritmos basados en Gradiente (Redes Neuronales, Regresión): El modelo tardará una eternidad en "converger" porque las actualizaciones de los pesos oscilarán de forma errática.

Algoritmos de Distancia (KNN, K-Means): El modelo simplemente ignorará las variables con rangos pequeños. Si buscas personas similares, el salario dominará la métrica y la edad no importará nada.

Modelos basados en árboles (Random Forest o XGBoost): son inmunes a la escala de los datos. A un árbol solo le importa si un valor es "mayor que" o "menor que", no qué tan lejos está.



Escalamiento de datos

“Si tengo dos columnas, una va desde 50 a 90 y la otra va desde 0 a 10000, si escalo de 0 a 1 las dos columnas, en todo caso, va a mantenerse la desproporción entre los dos features pero ahora entre 0 y 1, ¿no?”

- *La distribución interna se mantiene, solo que ahora en 0 y 1.*
- *Al escalar no cambiamos los datos en sí, sino la regla con la cual medimos. La columna de 10000 se medía con una regla de kilómetros y la columna de 50-90 con una regla de metros. Al escalar, ahora medimos con la misma regla.*



Escalamiento de datos

Min-Max Scaling (Normalization)

Comprime los datos a un rango fijo, típicamente $[0, 1]$.

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Cuando necesitas que los valores estén en un rango acotado y conocido , por ejemplo, en redes neuronales o en procesamiento de imágenes (píxeles 0-255 $\rightarrow$ 0-1).

Cuando la distribución no es necesariamente normal.

Es muy sensible a outliers, un solo valor extremo estira todo el rango y "aplasta" el resto de los datos hacia un rango muy pequeño.



Transformación de datos

Escalamiento de datos

Min-Max Scaling (Normalization)

Comprime los datos a un rango fijo, típicamente $[0, 1]$.

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Datos de entrenamiento
datos_train = pd.DataFrame({'edad': [20, 25, 30, 35, 40]})

scaler = MinMaxScaler()

# .fit() -> el scaler "aprende" el min y el max de los datos
scaler.fit(datos_train)

print("Min aprendido:", scaler.data_min_) # [20.]
print("Max aprendido:", scaler.data_max_) # [40.]

# .transform() -> aplica la fórmula usando el min/max ya aprendidos
datos_train_scaled = scaler.transform(datos_train)
print(datos_train_scaled)
```



Escalamiento de datos

Standar Scaler (Estandarization)

Transforma los datos usando la fórmula de **z-score**: resta la media y divide por la desviación estándar.

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

Donde:

- μ (mu) = media de la columna
- σ (sigma) = desviación estándar de la columna

Los datos transformados quedan con media = 0 y desviación estándar = 1, pero el rango no está acotado, pueden salir valores negativos, positivos, y en teoría cualquier magnitud (aunque en la práctica casi todo cae entre -3 y 3 si los datos son razonablemente normales).



Transformación de datos

Escalamiento de datos

Standar Scaler (Estandarization)

Transforma los datos usando la fórmula de **z-score**: resta la media y divide por la desviación estándar.

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
import pandas as pd

datos = pd.DataFrame({'edad': [20, 25, 30, 35, 40]})

scaler = StandardScaler()
scaler.fit(datos)

print("Media aprendida:", scaler.mean_)          # [30.]
print("Desv. estándar:", scaler.scale_)          # [7.07]

datos_scaled = scaler.transform(datos)
print(datos_scaled)
```

```
[[ -1.41]
 [ -0.71]
 [  0. ]
 [  0.71]
 [  1.41]]
```

Escalamiento de datos

RobustScaler.

Cuando tengamos outliers muy, muy extremos. En este caso se usa RobustScaler (mediana y cuartiles).

$$x_{scaled} = \frac{x - mediana}{IQR} = \frac{x - Q_2}{Q_3 - Q_1}$$



Transformación de datos

Ejemplos y ejercicios

